```
    private final List<JSONObject> links = new ArrayList<>
();    #A

    public Message(long spaceId, long msgId, String author,
        Instant time, String message) {
      this.spaceId = spaceId;
      this.msgId = msgId;
      this.author = author;
      this.time = time;
      this.message = message;
    }
    @Override
    public String toString() {
      JSONObject msg = new JSONObject();
      msg.put("uri",
          "/spaces/" + spaceId + "/messages/" + msgId);
      msg.put("author", author);
      msg.put("time", time.toString());
      msg.put("message", message);
      msg.put("links", links);    #B
      return msg.toString();
    }
  }
```

#A Add a list of link previews to the class
#B Return the links as a new field on the message response

Finally, you can update the `readMessage` method to scan the text of a message for strings that look like URLs and fetch a link preview for those links. You can use a regular expression to search for potential links in the message. In this case, you'll just look for any strings that start with http:// or https://, as shown in listing 10.14. Once a potential link has been found, you can use the `fetchLinkPreview` method you just wrote to fetch the link preview. If the link was valid and a